

Полная техническая документация проекта
IoT (ESP8266 + Firebase + React)

10 апреля 2026 г.

ПОЛНАЯ ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ ПРОЕКТА IOT (ESP8266 + FIREBASE + REACT)

СОДЕРЖАНИЕ

ПОЛНАЯ ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ ПРОЕКТА IOT (ESP8266 + FIREBASE + REACT)	1
СОДЕРЖАНИЕ.....	1
1 Назначение системы.....	3
1.1 Функциональные границы решения.....	3
1.2 Критерии качества отчётной системы.....	4
2 Состав репозитория	4
2.1 Логическая декомпозиция проекта	4
2.2 Принцип разделения ответственности	5
3 Архитектура на уровне подсистем.....	5
3.1 Поток данных и управляющий поток	5
4 Прошивка микроконтроллера (course_work)	6
4.1 Платформа и зависимости	6
4.2 Модульная структура прошивки	7
4.3 Роли модулей	7
4.4 Константы и интервалы (app_config).....	8
4.5 Инициализация (setup())	8
4.6 Причины выбранного порядка старта	9
4.7 Основной цикл (loop())	9
4.8 Логика активности пользователя (/current).....	10
4.9 Логика записи истории (/history)	10
4.10 Защита от невалидных датчиков	11
4.11 Отображение на OLED (display_service)	11
4.12 Вывод по прошивке	11
5 Облачный слой (Firebase Realtime Database)	11
5.1 Структура данных	11
5.2 Назначение веток в RTDB	12
5.3 Контракт данных.....	12
5.4 Преимущества выбранной модели хранения	12

6	Frontend (iot-dashboard).....	13
6.1	Стек.....	13
6.2	Архитектурный принцип frontend	13
6.3	Поток «живых» данных (useCurrentData)	13
6.4	Поток исторических данных (useHistoryData).....	14
6.5	UI-уровень	14
6.6	Обработка ошибок и устойчивость UI	15
6.7	Вывод по облачному и клиентскому уровням	15
7	Сквозные сценарии взаимодействия	15
7.1	Публикация текущих данных при активном пользователе	15
7.2	Почасовая запись истории	16
8	Машина состояний прошивки (упрощённо).....	16
8.1	Пояснение по состояниям	17
9	Важные инженерные особенности	17
10	Ограничения текущей реализации.....	17
11	Рекомендации по развитию	18
12	Карта соответствия «папка → ответственность»	18
13	Итоговое заключение.....	19

1 Назначение системы

Проект реализует полный IoT-контур для мониторинга параметров окружающей среды в реальном времени. Система предназначена для непрерывного сбора телеметрии, визуализации текущих значений, накопления исторических данных и поддержки удалённого контроля состояния объекта наблюдения.

С инженерной точки зрения решение закрывает четыре ключевые задачи:

1. **Сбор и первичная обработка телеметрии.** Микроконтроллер (ESP8266 / NodeMCU v2) считывает показания датчиков температуры, влажности и концентрации газа, выполняет базовую очистку данных и локально отображает значения на OLED-дисплее.
2. **Frontend (React + TypeScript)** читает данные из Firebase:
 - `current` — «живые» текущие показания;
 - `history` — почасовая история.
3. **Оптимизация трафика и нагрузки.** Frontend отправляет `heartbeat (/status/last_viewed)`, по которому прошивка понимает, активен ли пользователь, и решает, публиковать ли ветку `current`.
4. **Формирование аналитической базы.** Прошивка сохраняет агрегированную историю в Firebase по часовым срезам, что позволяет строить графики, анализировать тренды и оценивать динамику параметров без постоянной потоковой передачи.

1.1 Функциональные границы решения

В текущей реализации система ориентирована на телеметрию и визуализацию, без подсистемы управления исполнительными устройствами. Это сознательный выбор архитектуры: контур «измерение → хранение → отображение» изолирован от контура «управление → обратная связь», что снижает риск ошибок и упрощает сопровождение.

- **Входит в область решения:** сбор метрик, публикация актуальных данных, хранение истории, интерактивный просмотр в веб-интерфейсе.

- **Не входит в область решения:** аварийные команды на оборудование, пользовательские роли доступа, локальное долговременное хранилище на устройстве.

1.2 Критерии качества отчётной системы

Система оценивается по следующим базовым критериям:

- **Надёжность сбора данных:** цикл измерений выполняется периодически и не блокируется сетевыми операциями.
- **Согласованность времени:** все временные метки приводятся к UTC epoch, что исключает ошибки при интерпретации данных на разных клиентах.
- **Читаемость интерфейса:** пользователь получает как текущую картину, так и исторический контекст в удобном формате.
- **Масштабируемость по данным:** архитектура разделяет «live»-ветку и историческую ветку, снижая нагрузку при росте объёма измерений.

2 Состав репозитория

2.1 Логическая декомпозиция проекта

- `course_work/` — прошивка микроконтроллера (PlatformIO, Arduino framework). Здесь сосредоточены аппаратная инициализация, циклический сбор данных, синхронизация времени и обмен с Firebase.
- `iot-dashboard/` — веб-панель мониторинга (Vite + React + TypeScript + MUI + Recharts + Firebase SDK). Модуль отвечает за отображение текущих и исторических данных, пользовательские фильтры и heartbeat-активность.
- `generate/` — вспомогательные скрипты генерации данных (не участвуют в основном runtime-контуре прошивка↔frontend). Используются для подготовки тестовых/демо-наборов и отладки визуализаций.

2.2 Принцип разделения ответственности

Разделение по директориям выполнено в соответствии с принципом единой ответственности: микроконтроллерный код не содержит UI-логики, а frontend не содержит аппаратно-зависимых алгоритмов. Это позволяет независимо развивать обе части системы и упрощает тестирование.

3 Архитектура на уровне подсистем

На диаграмме показан полный путь телеметрии: от физического измерения до отображения на клиенте. Центральная роль у Firebase RTDB как у точки синхронизации между асинхронной прошивкой и реактивным веб-интерфейсом.

- Подсистема ESP8266 формирует данные и инициирует запись.
- Firebase хранит состояние и обеспечивает подписки/чтение.
- Frontend интерпретирует данные, отображает их и формирует heartbeat для управления жизненным циклом ветки `/current`.

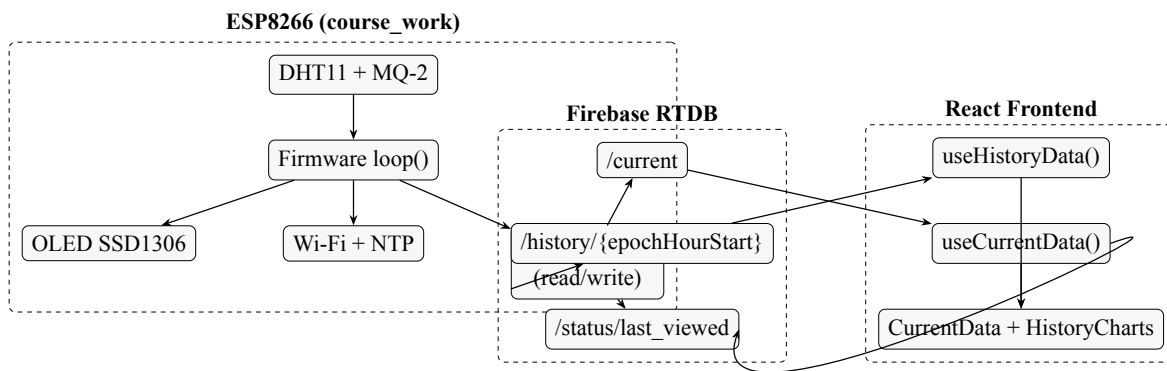


Рисунок 1 — Диаграмма зависимостей подсистем

3.1 Поток данных и управляющий поток

В архитектуре одновременно работают два связанных потока:

1. **Данные измерений:** датчики → прошивка → Firebase → frontend.

2. **Сигнал активности пользователя:** frontend → Firebase (/status/last_viewed) → прошивка.

Такой подход обеспечивает баланс между актуальностью данных и экономией ресурсов: система публикует «живую» ветку только в тот момент, когда это действительно нужно пользователю.

4 Прошивка микроконтроллера (course_work)

4.1 Платформа и зависимости

Файл конфигурации окружения: `course_work/platformio.ini`.

- Плата: `nodemcu2`.
- Фреймворк: `arduino`.
- Основные библиотеки:
 - `Adafruit SSD1306`, `Adafruit GFX` — рендеринг пользовательской информации на OLED;
 - `DHT sensor library` — работа с датчиком температуры и влажности DHT11;
 - `MQUnifiedsensor` — чтение и калибровка газового датчика MQ-2;
 - `NTPClient` — синхронизация сетевого времени;
 - `FirebaseClient (mobizt)` — асинхронные операции чтения/записи в RTDB.

Выбранный стек ориентирован на устойчивую работу в ограниченных аппаратных ресурсах ESP8266: каждая библиотека закрывает конкретный технологический риск (сенсоры, время, сеть, визуализация, облачная синхронизация).

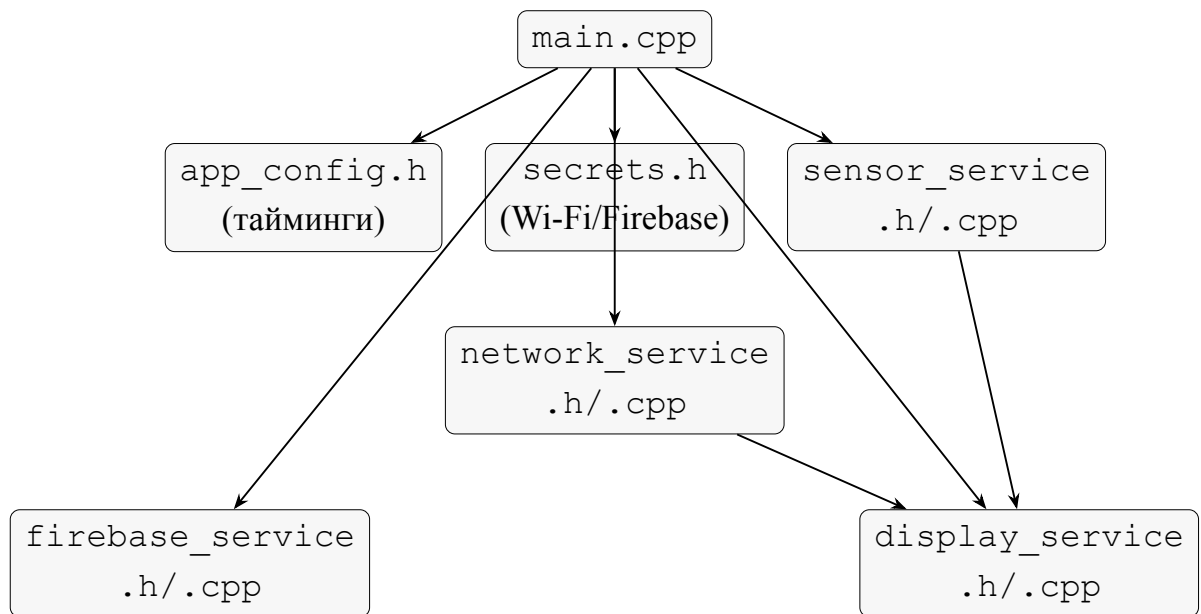


Рисунок 2 — Диаграмма зависимостей модулей прошивки

4.2 Модульная структура прошивки

4.3 Роли модулей

- `main.cpp` — точка входа, оркестрация старта, циклический диспетчер, общая координация сервисов.
- `app_config.h` — централизованный набор констант таймингов и порогов.
- `secrets.h` — параметры доступа к Wi-Fi и Firebase.
- `sensor_service` — чтение/калибровка датчиков, нормализация физических измерений.
- `network_service` — подключение к сети, поддержание времени и проверка валидности timestamp.
- `firebase_service` — публикация `/current`, запись `/history`, обработка активности пользователя.
- `display_service` — локальная визуализация текущего состояния и телеметрии.

4.4 Константы и интервалы (`app_config`)

- `WIFI_TIMEOUT_MS = 15000` — ограничение времени старта сетевого соединения.
- `SENSOR_UPDATE_INTERVAL_MS = 1000` — период обновления измерений и экрана.
- `ACTIVITY_INTERVAL_MS = 10000` — период проверки активности пользователя.
- `MIN_VALID_EPOCH = 1700000000` — минимально допустимое время после синхронизации.

Такая схема интервалов формирует иерархию частот: быстрый контур отвечает за измерения и UX, медленный контур — за облачную координацию и оптимизацию передачи.

4.5 Инициализация (`setup()`)

Порядок выполнения:

1. `Serial.begin(115200)`.
2. `beginDisplay()`:
 - при неуспехе — бесконечный цикл (жёсткий fail-stop).
3. `connectWiFiAndTime(SSID, PASSWORD, WIFI_TIMEOUT_MS)`:
 - успешное подключение к Wi-Fi;
 - запуск NTP (`pool.ntp.org`, offset 0, то есть UTC);
 - первичное обновление времени.
4. Если Wi-Fi подключён: `beginFirebase(FIREBASE_HOST)`.
5. `beginSensors()`:
 - `dht.begin()`;
 - калибровка MQ-2 (10 итераций, усреднение R0).
6. На OLED выводится `Init complete / Monitoring....`

4.6 Причины выбранного порядка старта

- **Display-first:** при проблемах на уровне индикации устройство сразу фиксирует отказ и не запускает «слепой» режим.
- **Network-before-cloud:** инициализация Firebase имеет смысл только при наличии сети и валидной временной базы.
- **Sensors-after-core-services:** к моменту измерений уже готовы журналирование, экран и сетевые подсистемы.

4.7 Основной цикл (`loop()`)

Непрерывная часть

- `loopFirebase()` вызывается всегда, чтобы обрабатывать асинхронные ответы и завершённые облачные операции без блокировки цикла.

Периодическая часть (каждые 1 секунду)

1. `updateNetworkTime()`.
2. Чтение датчиков: `readSensors()`:
 - `temperature = dht.readTemperature();`
 - `humidity = dht.readHumidity();`
 - `gas = mq2.readSensor();`
3. Получение времени:
 - `epoch = getNetworkEpoch();`
 - `formattedTime = getNetworkFormattedTime();`
 - `timeValid = isNetworkTimeValid(epoch, MIN_VALID_EPOCH);`
4. Отрисовка OLED: `renderTelemetry(...)`.
5. При выполнении условий (Wi-Fi + Firebase ready + valid time):
 - а) раз в 10 секунд — `checkActivityAndSyncCurrent(...)` и чтение `/status/last_viewed;`

- б) раз в час (в первые 60 секунд) — запись истории в `/history/{hourStartEpoch}`.

4.8 Логика активности пользователя (`/current`)

Ключевая идея: ветка `/current` существует только при подтверждённой пользовательской активности.

1. Прошивка сохраняет `pending`-значения телеметрии и выполняет `get("/status/last_viewed")`.
2. После завершения запроса в `loopFirebase()` вычисляется `diff = |pendingEpoch - last_viewed|`.
3. Если `diff < 120` секунд:
 - выполняется запись `/current = {"t":..., "h":..., "g":...}`.
4. Иначе:
 - выполняется удаление узла `/current`.

Механизм уменьшает объём «пустого» realtime-трафика и сохраняет поведение интуитивным: пользователь видит «живые» данные только тогда, когда интерфейс действительно активен.

4.9 Логика записи истории (`/history`)

- Частота: **1 запись в час**.
- Защита от дубликатов: `lastHistoryHourSent`.
- Формат пути: `/history/<epoch начала часа UTC>`.
- Payload: `t, h, g`.

Почасовая дискретизация обеспечивает компромисс между детализацией и объёмом хранилища, достаточный для практического анализа динамики на уровне часов и суток.

4.10 Защита от невалидных датчиков

Перед отправкой:

- `temperature/humidity` проходят `sanitizeReading()`:
 - если NaN, заменяются на 0.0.
- Газ отправляется как есть (округление до ppm выполняется на frontend).

На OLED:

- при NaN у температуры/влажности отображается 0.

4.11 Отображение на OLED (`display_service`)

- Разрешение: 128×32.
- Обычный режим:
 - строка времени (при валидном NTP);
 - строки Tmp, Hum, Gas.
- Режим ожидания синхронизации:
 - отображается `Time: Syncing....`

4.12 Вывод по прошивке

Прошивка построена как неблокирующая сервисная система с явным разделением быстрых и медленных циклов. Это обеспечивает устойчивую телеметрию, предсказуемое поведение UI и контролируемую нагрузку на облако даже при нестабильной сети.

5 Облачный слой (Firebase Realtime Database)

5.1 Структура данных

```
1 {  
2   "current": { "t": 23.5, "h": 62.1, "g": 3200 },  
3   "history": {  
4     "1712700000": { "t": 22.1, "h": 64.2, "g": 2800 },
```

```

5     "1712703600": { "t": 22.5, "h": 63.8, "g": 2900 }
6 },
7 "status": {
8     "last_viewed": 1712703650
9 }
10 }

```

5.2 Назначение веток в RTDB

- `/current` — оперативное «live»-состояние для мгновенного отображения в UI.
- `/history/<hourEpoch>` — архив телеметрии с часовым шагом.
- `/status/last_viewed` — технический маркер активности клиентского интерфейса.

В коде фронтенда предусмотрен парсинг и объекта, и JSON-строки, поэтому система устойчива к различным форматам payload в базе.

5.3 Контракт данных

- `t`: температура, `number`, °C.
- `h`: влажность, `number`, %.
- `g`: газ, `number` (сырой сигнал датчика, в UI приводится к ppm округлением).
- Все timestamp — **Unix epoch, секунды, UTC**.

5.4 Преимущества выбранной модели хранения

1. **Разделение горячих и холодных данных:** ветка `/current` не перегружает историю, а `/history` не мешает быстрому отображению.
2. **Простота клиентской подписки:** для live-потока используется один наблюдаемый путь.
3. **Предсказуемая ретроспектива:** исторические точки имеют фиксированную структуру ключей и легко сортируются.

6 Frontend (iot-dashboard)

6.1 Стек

- React 19 + TypeScript.
- Vite.
- Material UI.
- Recharts.
- Firebase JS SDK (Realtime Database).

6.2 Архитектурный принцип frontend

Frontend построен по схеме «данные в хуках, отображение в компонентах». Это уменьшает связанность кода: бизнес-логика доступа к RTDB не смешивается с визуальным представлением.

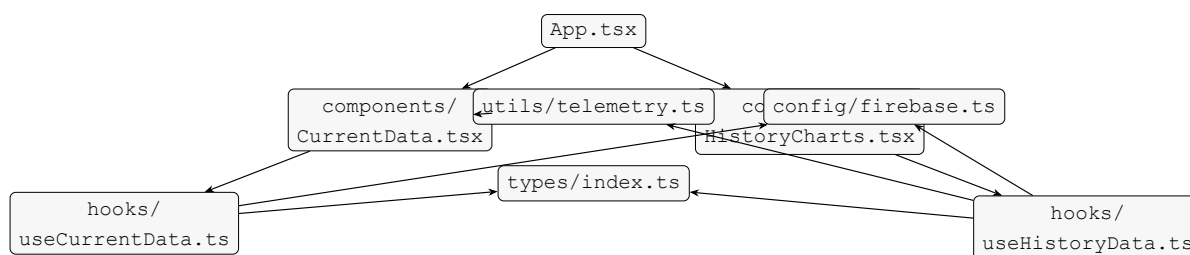


Рисунок 3 — Диаграмма зависимостей модулей frontend

6.3 Поток «живых» данных (useCurrentData)

1. Создаётся подписка `onValue(ref(database, "current"))`.
2. На каждом обновлении:
 - `payload` проходит `parseTelemetryPayload`;
 - при успехе обновляется `state data, lastUpdated`;
 - при ошибке обновляется `state error`.
3. Параллельно работает `heartbeat`:
 - сразу при монтировании;
 - затем каждые 30 сек;

- в `/status/last_viewed` пишется текущее UTC-время клиента.

Heartbeat — это управляющий сигнал для прошивки. Благодаря ему MCU понимает, нужен ли пользователю live-поток прямо сейчас.

6.4 Поток исторических данных (`useHistoryData`)

1. По кнопке `Load data` компонент применяет диапазон дат.
2. Хук делает `get(ref(database, "history"))`.
3. Клиент фильтрует записи по диапазону `startTimestamp..endTimestamp`.
4. Каждая запись:
 - валидируется и парсится;
 - преобразуется в `ChartDataPoint`;
 - сортируется по `timestamp`.
5. `HistoryCharts` строит график в выбранном режиме:
 - все метрики;
 - только температура;
 - только влажность;
 - только газ.

6.5 UI-уровень

- `CurrentData`:
 - 3 карточки (`temperature/humidity/gas`);
 - индикатор режима `Live`;
 - время последнего обновления.
- `HistoryCharts`:
 - выбор диапазона дат;
 - валидация диапазона;
 - интерактивная визуализация на `Recharts`.

6.6 Обработка ошибок и устойчивость UI

- Ошибки чтения не приводят к остановке приложения, а отображаются пользователю в виде контролируемого состояния.
- Невалидные payload не ломают компонент, а отсекаются на этапе парсинга.
- Отсутствие `/current` корректно интерпретируется как неактивный поток, а не как критическая ошибка.

6.7 Вывод по облачному и клиентскому уровням

Связка Firebase + React-хуки обеспечивает реактивную модель отображения с чётким разделением ответственности: RTDB хранит и синхронизирует состояние, hooks инкапсулируют доступ к данным, компоненты фокусируются на UX и визуальной достоверности представления.

7 Сквозные сценарии взаимодействия

7.1 Публикация текущих данных при активном пользователе

```
1 sequenceDiagram
2   participant FE as Frontend
3   participant FB as Firebase
4   participant MCU as ESP8266
5
6   FE->>FB: set /status/last_viewed = now (каждые 30с)
7   MCU->>FB: get /status/last_viewed (каждые 10с)
8   FB-->>MCU: last_viewed
9   MCU->>MCU: diff = |epoch - last_viewed|
10  alt diff < 120s
11    MCU->>FB: set /current = {t,h,g}
12    FB-->>FE: onValue(/current) update
13  else diff >= 120s
14    MCU->>FB: remove /current
15    FB-->>FE: /current = null
16  end
```


Инженерный смысл сценария

Сценарий реализует адаптивную передачу данных. Если пользователь недавно взаимодействовал с интерфейсом, система поддерживает активный live-поток. Если пользователь неактивен, поток отключается автоматически, что уменьшает число операций записи/чтения и экономит ресурсы сети.

7.2 Почасовая запись истории

```
1 sequenceDiagram
2   participant MCU as ESP8266
3   participant FB as Firebase
4   participant FE as Frontend
5
6   MCU->>MCU: Каждый цикл проверка epoch % 3600 < 60
7   MCU->>FB: set /history/{hourStartEpoch} = {t,h,g}
8   FE->>FB: get /history (по кнопке Load data)
9   FE->>FE: фильтрация по диапазону и сортировка
10  FE->>FE: рендер графиков
```

Инженерный смысл сценария

Исторический поток отделён от live-потока и имеет другую частоту. Это предотвращает разрастание базы «мелкими» записями и сохраняет достаточную аналитическую ценность для графиков и отчётности.

8 Машина состояний прошивки (упрощённо)

```
1 stateDiagram-v2
2   [*] --> Boot
3   Boot --> DisplayInit
4   DisplayInit --> WifiTry
5   WifiTry --> WifiOk: connected
6   WifiTry --> Offline: timeout
7   WifiOk --> FirebaseInit
8   FirebaseInit --> SensorInit
9   Offline --> SensorInit
10  SensorInit --> Monitoring
11  Monitoring --> Monitoring: read sensors + render OLED
12  Monitoring --> CloudSync: Wi-Fi && FirebaseReady && validTime
```

8.1 Пояснение по состояниям

- **Boot / DisplayInit** — аппаратная готовность и первичная верификация канала локальной индикации.
- **WifiTry / WifiOk / Offline** — развилка сетевой доступности без блокировки основного назначения устройства.
- **FirebaseInit** — подготовка облачного канала только при наличии сетевой инфраструктуры.
- **SensorInit / Monitoring** — переход к целевому рабочему режиму наблюдения.
- **CloudSync** — условный контур облачного обмена, активируемый только при валидных предусловиях.

9 Важные инженерные особенности

1. **UTC как единая временная шкала:** NTP offset = 0, timestamps хранятся в epoch-секундах.
2. **Асинхронный Firebase-контур:** сетевые операции не блокируют цикл измерений и индикации.
3. **Событийное управление live-поток:** /current зависит от heartbeat-сигнала пользователя.
4. **Экономичный исторический контур:** архив формируется батчево и запрашивается по требованию.
5. **Явный fail-fast на OLED:** в текущей конфигурации отсутствие дисплея считается критическим отказом.

10 Ограничения текущей реализации

1. **Секреты в secrets.h:** учётные данные хранятся в коде и требуют усиления политики безопасности.

2. **Чтение всей ветки истории:** при росте объёма данных возрастает нагрузка на сеть и клиентскую фильтрацию.
3. **Отсутствие device-auth:** модель NoAuth снижает уровень защищённости подключения устройства.
4. **Жёсткая реакция на сбой экрана:** отсутствует деградированный режим работы без OLED.

11 Рекомендации по развитию

1. Вынести секреты в приватный конфиг и исключить из репозитория.
2. Добавить аутентификацию устройства (custom token через доверенный backend).
3. Перейти к диапазонным запросам истории и индексированию по времени.
4. Внедрить буферизацию оффлайн-измерений с последующей отправкой после восстановления сети.
5. Добавить телеметрию состояния устройства: uptime, RSSI, ошибки публикации, статистику ретраев.

12 Карта соответствия «папка → ответственность»

Путь	Ответственность
course_work/src/main.cpp	orchestration boot + runtime loop
course_work/src/sensor_service.cpp	чтение DHT11 и MQ-2, калибровка MQ-2
course_work/src/network_service.cpp	Wi-Fi + NTP + валидация времени
course_work/src/firebase_service.cpp	async read/write Firebase, activity logic
course_work/src/display_service.cpp	визуализация статусов и телеметрии на OLED
iot-dashboard/src/hooks/useCurrentData.ts	realtime-подписка на current + heartbeat

<code>iot-dashboard/src/hooks/useHistoryData.ts</code>	загрузка и фильтрация history
<code>iot-dashboard/src/components/CurrentData.tsx</code>	карточки текущих показаний
<code>iot-dashboard/src/components/HistoryCharts.tsx</code>	графики истории и фильтры диапазона
<code>iot-dashboard/src/utils/telemetry.ts</code>	парсинг payload и нормализация газа

13 Итоговое заключение

Реализованная система образует завершённый технический контур IoT-мониторинга: устройство стабильно собирает телеметрию, облачный слой согласованно хранит и синхронизирует данные, а frontend предоставляет пользователю актуальную и историческую картину состояния среды.

Ключевое достоинство архитектуры — предсказуемое поведение под нагрузкой за счёт разделения потоков данных, асинхронного обмена и строгой временной модели на базе UTC epoch.